CHAPTER 13

Route 53 and ALB

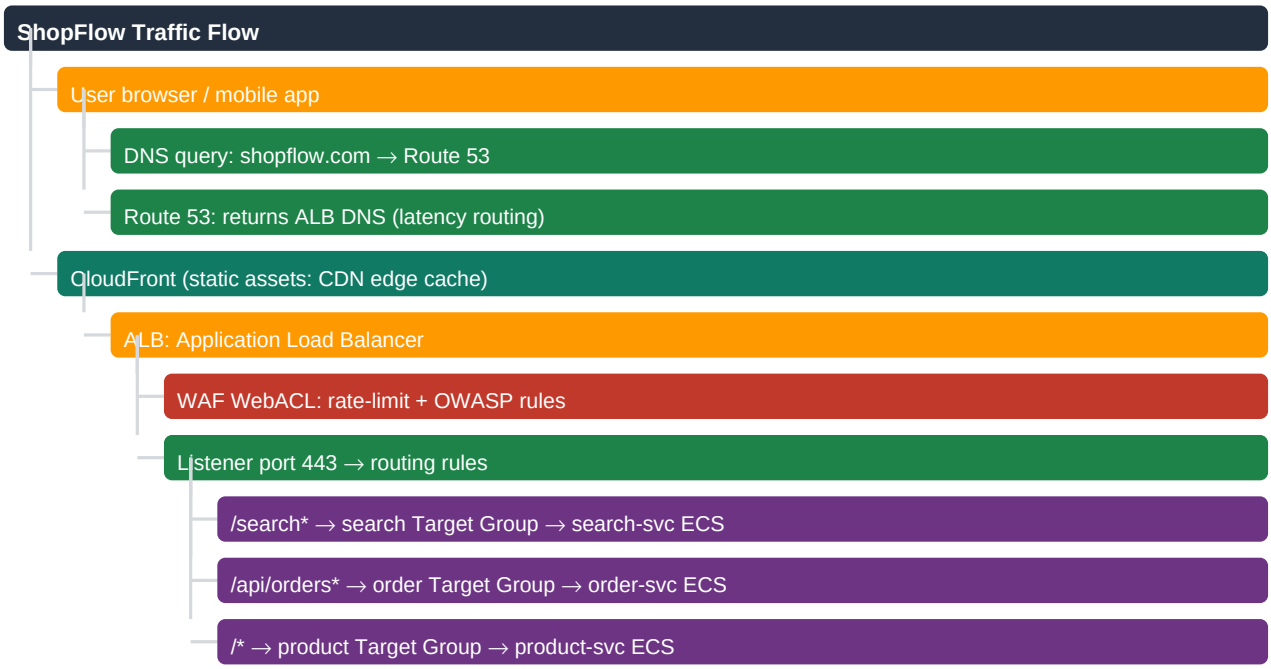
DNS Routing · Health Checks · Listener Rules · WAF · TLS · Failover

AWS Tutorial · ShopFlow

13-00 Chapter Overview — Traffic Management at ShopFlow

ShopFlow receives 1,000 req/s normal and 10,000 req/s Black Friday. Traffic flows through Route 53 (DNS + health checks + routing policies) → Application Load Balancer (TLS termination + host/path routing + WAF) → ECS Fargate tasks. Route 53 and ALB together provide high availability, geo-routing, and blue/green deployments.

Layer	Service	ShopFlow Usage	Capability
DNS Resolution	Route 53	shopflow.com, api.shopflow.com, search.shopflow.com	Latency routing, health checks, failover, weighted A/B
TLS Termination	ALB + ACM Certificate	*.shopflow.com wildcard cert	HTTPS offload, HTTP/2, connection keep-alive
Path-based Routing	ALB Listener Rules	/ → product-svc, /search → search-svc, /api/orders → order-svc	Microservice routing without separate load balancers per serv
WAF Integration	WAF WebACL on ALB	Rate limiting (10K req/5min per IP), SQL injection, OWASP Top 10 rules	Block malicious traffic at edge before hitting ECS
Health Checks	Route 53 + ALB	/actuator/health on each ECS task, 30s interval, 3 failures	Remove unhealthy targets; failover to healthy AZ



13-01 Route 53 Hosted Zone and Record Types

Route 53 hosts the shopflow.com domain. DNS records: A record (alias to ALB, no TTL charge), CNAME for subdomains, MX for email, TXT for SPF/DKIM. Alias records (not CNAME) are required for root domain (shopflow.com) pointing to AWS resources — CNAMEs cannot be used at the zone apex.

Route 53 CDK — Hosted Zone and Records

```
const hostedZone = route53.HostedZone.fromLookup(this,"ShopFlowZone",{
domainName: "shopflow.com",
});
```

```
// A record: shopflow.com → ALB (alias record — no TTL, no extra charge)
new route53.ARecord(this,"ApexAlias",{
zone: hostedZone,
recordName: "shopflow.com",
target: route53.RecordTarget.fromAlias(
new targets.LoadBalancerTarget(alb)),
});
// CNAME: api.shopflow.com → ALB DNS name
new route53.CnameRecord(this,"ApiCname",{
zone: hostedZone,
recordName: "api",
domainName: alb.loadBalancerDnsName,
ttl: Duration.seconds(60),
});
// A record: search.shopflow.com → same ALB (different listener rule routes /search)
new route53.ARecord(this,"SearchAlias",{
zone: hostedZone,
recordName: "search",
target: route53.RecordTarget.fromAlias(
new targets.LoadBalancerTarget(alb)),
});
```

Record Type	Format	TTL Allowed	Use Case	ShopFlow Example
A (Alias)	→ AWS resource (ALB, CloudFront, S3)	No TTL (AWS sets)	Root domain, apex alias	shopflow.com → ALB
CNAME	hostname → hostname	Required (60s)	Subdomains to external endpoints	www.shopflow.com → shopflow.com
MX	priority + mail server	300s	Email routing	shopflow.com MX → ses.amazonaws.com
TXT	text string	300s	SPF, DKIM, domain ownership verification	v=spf1 include:amazonses.com ~all
NS	nameserver hostnames	172800s	Delegate subdomain to another zone	Not used (all in single hosted zone)

13-02 Route 53 Routing Policies

Route 53 offers 7 routing policies. ShopFlow uses: Latency routing (return ALB closest to user), Weighted routing (10% traffic to canary ALB for A/B testing), and Failover routing (primary: us-east-1, secondary: us-west-2 for DR). Each policy is set per DNS record.

Routing Policy	How It Works	ShopFlow Use Case	When to Use
Simple	Single resource returned	Not used (would bypass HA)	Single resource, no health checks
Weighted	% split across multiple records	10%/90% A/B canary deployment	Gradual traffic shifts, A/B testing
Latency	Returns lowest-latency region for caller	Multi-region: us-east-1 + us-west-2	Minimize latency for global users
Failover	Primary returns healthy; Secondary if primary fails	us-east-1 primary; us-west-2 secondary	Active-passive DR across regions
Geolocation	Routes by caller geographic location	Block non-US traffic (regulatory)	Data sovereignty, content licensing
Geoproximity	Routes by distance with bias factor	Not used (latency routing preferred)	Fine-grained geographic control
Multi-Value Answer	Returns up to 8 healthy records	Not used (ALB handles this)	Client-side load balancing

Weighted Routing CDK — Canary Deployment

```
// 90% to production ALB, 10% to canary ALB
new route53.ARecord(this,"ProductionWeight",{
zone: hostedZone, recordName: "api",
target: route53.RecordTarget.fromAlias(new targets.LoadBalancerTarget(productionAlb)),
weight: 90,
setIdentifier: "production",
});
```

```
new route53.ARecord(this,"CanaryWeight",{
zone: hostedZone, recordName: "api",
target: route53.RecordTarget.fromAlias(new targets.LoadBalancerTarget(canaryAlb)),
weight: 10,
setIdentifier: "canary",
});
```

13-03 Route 53 Health Checks

Route 53 health checks probe endpoints every 10-30 seconds from 15 global health checkers. A record is removed from DNS if the endpoint is unhealthy. ShopFlow configures health checks on the ALB endpoint (/actuator/health) with a 3-failure threshold (30s total for 10s interval).

Health Check CDK

```
const albHealthCheck = new route53.CfnHealthCheck(this,"AlbHealthCheck",{
healthCheckConfig: {
type: "HTTPS",
fullyQualifiedDomainName: alb.loadBalancerDnsName,
port: 443,
resourcePath: "/actuator/health",
requestInterval: 10, // check every 10 seconds
failureThreshold: 3, // 3 consecutive failures = unhealthy (30s RTO)
measureLatency: true, // enables CloudWatch HealthCheckPercentageHealthy metric
regions: [ // check from 3 continents for geographic coverage
"us-east-1","eu-west-1","ap-southeast-1"
],
},
});
// Attach health check to failover routing record
new route53.ARecord(this,"PrimaryFailover",{
zone: hostedZone, recordName: "api",
target: route53.RecordTarget.fromAlias(new targets.LoadBalancerTarget(alb)),
setIdentifier: "primary",
healthCheck: albHealthCheck,
failover: route53.RecordType.PRIMARY,
});
```

Health Check Type	Protocol	Monitors	Route 53 Action
HTTPS endpoint (ShopFlow)	HTTPS GET to /actuator/health	HTTP 200 response; body can be checked	Remove DNS record if unhealthy; restore when healthy
String match	HTTPS + response body check	URL + "status:UP" in body	Stricter: body must contain expected string
Calculated health check	Logical combination of other checks	All child checks must pass (AND logic)	Use for: ALB healthy AND database healthy
CloudWatch alarm state	CloudWatch metric alarm state	Alarm = unhealthy, OK = healthy	For non-HTTP checks: e.g., SQS queue depth alarm

13-04 Application Load Balancer Setup

ShopFlow ALB: internet-facing, deployed in public subnets, TLS termination with ACM wildcard certificate. ALB replaces EC2 per-service load balancers with a single ALB handling all microservice routing via listener rules. Connection pooling at ALB (HTTP/2, keep-alive) reduces TCP handshake overhead.

ALB CDK

```
const alb = new elbv2.ApplicationLoadBalancer(this,"ShopFlowAlb",{
vpc,
internetFacing: true,
vpcSubnets: {subnetType: ec2.SubnetType.PUBLIC},
securityGroups: [sgAlb],
http2Enabled: true,
idleTimeout: Duration.seconds(60),
```

```
deletionProtection: true, // prevent accidental delete
});
// HTTP → HTTPS redirect (port 80 listener)
alb.addListener("Http",{
port: 80,
defaultAction: elbv2.ListenerAction.redirect({
protocol: "HTTPS", port: "443", permanent: true,
}),
});
// HTTPS listener with ACM certificate
const httpsListener = alb.addListener("Https",{
port: 443,
certificates: [elbv2.ListenerCertificate.fromArn(certArn)],
sslPolicy: elbv2.SslPolicy.TLS13_RES, // TLS 1.3 only; rejects TLS 1.2
defaultAction: elbv2.ListenerAction.fixedResponse(404,{
contentType: "application/json",
messageBody: JSON.stringify({error:"Not found"}),
}),
});
```

■ Use SslPolicy.TLS13_RES (TLS 1.3 restricted) for all new ALBs. TLS 1.3 is faster (1-RTT handshake vs 2-RTT for TLS 1.2), more secure (forward secrecy mandatory), and removes weak cipher suites. If you have legacy clients (old Android <7.0, IE11) that cannot use TLS 1.3, use TLS12_EXT which supports both TLS 1.2 and 1.3. For ShopFlow, mobile/web clients all support TLS 1.3.

13-05 ALB Listener Rules — Microservice Routing

ALB listener rules match requests by host header, path prefix, HTTP method, query string, or source IP. Rules are evaluated in priority order (lowest number = highest priority). ShopFlow uses path-based routing to route requests to the correct microservice target group without separate load balancers per service.

Listener Rules CDK

```
// Rule 1: /search/* → search-svc Target Group (highest priority)
httpsListener.addTargetGroups("SearchRule",{
priority: 10,
conditions: [elbv2.ListenerCondition.pathPatterns(["/search/*", "/suggest/*"])],
targetGroups: [searchTg],
});
// Rule 2: /api/orders/* → order-svc Target Group
httpsListener.addTargetGroups("OrderRule",{
priority: 20,
conditions: [elbv2.ListenerCondition.pathPatterns(["/api/orders/*", "/api/returns/*"])],
targetGroups: [orderTg],
});
// Rule 3: /api/cart/* → cart-svc Target Group
httpsListener.addTargetGroups("CartRule",{
priority: 30,
conditions: [elbv2.ListenerCondition.pathPatterns(["/api/cart/*"])],
targetGroups: [cartTg],
});
// Rule 4: /api/users/* → user-svc (Cognito auth required)
httpsListener.addAction("UserRule",{
priority: 40,
conditions: [elbv2.ListenerCondition.pathPatterns(["/api/users/*"])],
action: elbv2.ListenerAction.authenticateCognito({
userPool, userPoolClient, userPoolDomain,
next: elbv2.ListenerAction.forward([userTg]),
}),
});
// Default: /* → product-svc (catch-all)
```

Rule Priority	Path Pattern	Target Group	Notes
10	/search/*, /suggest/*	searchTg → search-svc	OpenSearch-backed full-text search
20	/api/orders/*, /api/returns/*	orderTg → order-svc	Order CRUD; write to Aurora
30	/api/cart/*	cartTg → cart-svc	Redis-backed cart; stateless ECS tasks
40	/api/users/*	userTg → user-svc	Cognito auth required at ALB layer
Default	/* (all other paths)	productTg → product-svc	Product catalog browse, homepage

13-06 Target Groups and Health Checks

ALB target groups contain the ECS Fargate task IPs registered dynamically by ECS. Each target group has its own health check (HTTP GET to /actuator/health). Unhealthy targets are automatically removed from rotation. Deregistration delay (30s) allows in-flight requests to complete before task shutdown.

Target Group CDK

```
const productTg = new elbv2.ApplicationTargetGroup(this,"ProductTg",{
vpc,
port: 8080,
protocol: elbv2.ApplicationProtocol.HTTP,
targetType: elbv2.TargetType.IP, // ECS Fargate uses IP-based targets
healthCheck: {
path: "/actuator/health",
healthyHttpCodes: "200",
interval: Duration.seconds(15),
timeout: Duration.seconds(5),
healthyThresholdCount: 2, // 2 consecutive successes = healthy
unhealthyThresholdCount: 3, // 3 consecutive failures = unhealthy (45s)
},
deregistrationDelay: Duration.seconds(30), // drain in-flight requests
stickinessCookieDuration: Duration.days(1), // sticky sessions (optional)
});
```

Health Check Param	ShopFlow Value	Too Low	Too High
interval	15 seconds	May overwhelm /health endpoint with many tasks	Slow detection of unhealthy tasks
timeout	5 seconds	False unhealthy on slow startup	Masks genuine slow responses
healthyThreshold	2 checks	Flapping: task oscillates healthy/unhealthy	Slow recovery after transient failure
unhealthyThreshold	3 checks (45s RTO)	Too aggressive: brief GC pause causes unhealthy	Failed task receives requests too l
deregistrationDelay	30 seconds	In-flight requests aborted (HTTP 502)	Slow deployments (waits unnecessarily)

13-07 ALB Access Logs and Request Tracing

ALB access logs record every request (source IP, latency, target IP, status code, user agent). Logs delivered to S3 every 5 minutes. ShopFlow uses access logs for: per-endpoint latency analysis, bot detection, SLA compliance, and feeding OpenSearch Ingestion pipeline (Ch12-14). X-Amzn-Trace-Id header traces requests end-to-end via X-Ray.

ALB Access Logs CDK

```
// Enable access logs to S3
alb.logAccessLogs(logBucket, "alb-access-logs");
// S3 lifecycle: keep 90 days (then delete or Glacier)
logBucket.addLifecycleRule({
prefix: "alb-access-logs/",
expiration: Duration.days(90),
transitions: [{
```

```
storageClass: s3.StorageClass.INFREQUENT_ACCESS,
transitionAfter: Duration.days(30),
}],
});
// ALB access log format (sample fields):
// type timestamp elb client:port target:port request_processing_time
// target_processing_time response_processing_time elb_status_code
// target_status_code received_bytes sent_bytes "request" "user_agent"
// ssl_cipher ssl_protocol target_group_arn "trace_id" ...
// Analyze with Athena (query S3 access logs):
// SELECT elb_status_code, COUNT(*) as count,
// AVG(target_processing_time) as avg_latency_s
// FROM alb_access_logs
// WHERE from_iso8601_timestamp(time) > current_timestamp - interval '1' hour
// GROUP BY elb_status_code ORDER BY count DESC
```

■ X-Amzn-Trace-Id header is added by ALB to every request. Format: Root=1-5f0xxx-abc;Self=1-5f0xxx-def. Spring Boot (with X-Ray SDK) reads this header, creates a trace segment, and propagates it to downstream services via HTTP headers. X-Ray console shows the full call chain: ALB → product-svc → Aurora → ElastiCache as a service map with latency breakdown at each hop. Enable ALB X-Ray integration: alb.setAttribute("routing.http.x_amzn_tls_version_and_cipher_suite.enabled","true").

13-08 WAF Integration with ALB

AWS WAF (Web Application Firewall) WebACL is attached to the ShopFlow ALB. WAF evaluates rules before requests reach ECS tasks. Rules: rate limiting (10,000 req/5min per IP), AWS Managed Rules (OWASP Top 10, known bad IPs, bot control), and custom rules (block scraper user agents).

WAF WebACL CDK

```
const wafAcl = new wafv2.CfnWebACL(this,"ShopFlowWaf",{
scope: "REGIONAL", // REGIONAL for ALB; CLOUDFRONT for CloudFront distributions
defaultAction: {allow: {}},
visibilityConfig: {
cloudWatchMetricsEnabled: true,
metricName: "ShopFlowWAF",
sampledRequestsEnabled: true,
},
rules: [
{
name: "RateLimit",
priority: 1,
action: {block: {}},
statement: {
rateBasedStatement: {
limit: 10000, // max 10K req per 5 minutes per IP
aggregateKeyType: "IP",
},
},
visibilityConfig: {cloudWatchMetricsEnabled:true,
metricName:"RateLimit",sampledRequestsEnabled:true},
},
{
name: "AWSManagedRulesCommonRuleSet",
priority: 2,
statement: {managedRuleGroupStatement: {
vendorName: "AWS", name: "AWSManagedRulesCommonRuleSet"}},
overrideAction: {none: {}},
visibilityConfig: {cloudWatchMetricsEnabled:true,
metricName:"CommonRules",sampledRequestsEnabled:true},
}
],
```

```
});
new wafv2.CfnWebACLAssociation(this,"WafAlbAssoc",{
resourceArn: alb.loadBalancerArn,
webAclArn: wafAcl.attrArn,
});
```

13-09 ACM Certificate and TLS Management

AWS Certificate Manager (ACM) provisions and renews TLS certificates for free. ShopFlow uses a wildcard certificate (*.shopflow.com) covering all subdomains. Certificate is DNS-validated (ACM adds a CNAME to Route 53 for auto-renewal). ALB handles TLS termination; backend ECS tasks communicate over HTTP.

ACM Certificate CDK

```
const cert = new acm.Certificate(this,"ShopFlowCert",{
domainName: "shopflow.com",
subjectAlternativeNames: ["*.shopflow.com"], // covers api., search., www.
validation: acm.CertificateValidation.fromDns(hostedZone), // auto DNS validation
});
// ACM auto-adds CNAME to Route 53 for validation:
// _abc123.shopflow.com CNAME _xyz789.acm-validations.aws
// Once validated, ACM renews the cert automatically 60 days before expiry
// No manual renewal required — CDK creates the cert and Route 53 record together
```

TLS Config	ShopFlow Setting	Reason
Certificate type	Wildcard (*.shopflow.com)	Single cert covers all subdomains — no per-service certs
Validation method	DNS (Route 53 CNAME)	Auto-renews without human intervention
TLS policy	TLS13_RES (TLS 1.3 only)	Most secure; 1-RTT handshake; forward secrecy
Termination point	ALB (not EC2/ECS)	TLS offloaded at ALB; backends use HTTP internally
Backend protocol	HTTP (ALB → ECS on port 8080)	Traffic stays inside VPC; encryption by VPC firewall
Certificate storage	ACM (never on EC2 or ECS)	No private key exposure; ACM handles key storage

13-10 Multi-Region Failover Architecture

ShopFlow DR: us-east-1 (primary) → us-west-2 (secondary). Aurora Global Database replicates writes from us-east-1 to us-west-2 with <1s lag. Route 53 failover routing: if us-east-1 ALB health check fails for 30s, Route 53 automatically routes traffic to us-west-2 ALB. RTO: 5 minutes (Aurora failover + ECS scale-up in us-west-2).

Route 53 Failover CDK

```
// Primary record: us-east-1 ALB with health check
new route53.ARecord(this,"PrimaryRecord",{
zone: hostedZone,
recordName: "api",
target: route53.RecordTarget.fromAlias(new targets.LoadBalancerTarget(primaryAlb)),
setIdentifier: "primary",
failover: route53.RecordType.PRIMARY,
healthCheck: primaryHealthCheck,
});
// Secondary record: us-west-2 ALB (no health check required on secondary)
new route53.ARecord(this,"SecondaryRecord",{
zone: hostedZone,
recordName: "api",
target: route53.RecordTarget.fromAlias(new targets.LoadBalancerTarget(secondaryAlb)),
setIdentifier: "secondary",
failover: route53.RecordType.SECONDARY,
});
```

Scenario	Route 53 Response	RTO	Data Loss (RPO)
Normal operation	Primary (us-east-1 ALB)	N/A	N/A
us-east-1 ALB unhealthy (ECS failure)	Failover to us-west-2 after 30s	30s DNS propagation + ~60s (Aurora TTL	None (Aurora Global — <1s lag)
us-east-1 AZ failure (1 AZ)	No failover — ALB routes to healthy AZs	~30s (ALB health check removal	None
us-east-1 region failure	Failover to us-west-2	~5 min (Aurora promote + ECS scale-up)	Seconds (Aurora Global replication lag)
Recovery (us-east-1 restored)	Manual fail-back via Route 53 weight update	Operator-controlled	None (replay from us-west-2 Aurora Global

13-11 ALB Sticky Sessions and Connection Draining

Sticky sessions (session affinity) route a user's requests to the same ECS task using a cookie. ShopFlow avoids sticky sessions by storing all session state in ElastiCache Redis (stateless ECS tasks). However, cart-svc uses duration-based stickiness as a temporary optimization to reduce Redis lookups for active shopping sessions.

Session Strategy	Mechanism	Pros	Cons	ShopFlow Usage
No stickiness (default)	ALB distributes round-robin	Perfect load distribution; any task handles any request	All state must be in external store (Redis/DB)	product-svc, order-svc, search-
Application-based sticky	ECS task sets AWSALB cookie	App controls stickiness duration	Tasks must be homogeneous; ALB re	None — ALB route changes if task ren
Duration-based sticky	ALB sets AWSALB cookie (1-day TTL)	Simple; no app code changes	Uneven distribution if users have long sessions	cart-svc: 1-day TTL
No sticky (Redis-backed)	State in Redis; no ALB cookie	Any task handles any user	Redis latency for every request (~1ms)	Recommended default

13-12 Private ALB for Internal Microservices

ShopFlow uses a separate internal (private) ALB for service-to-service communication. product-svc calls inventory-svc and pricing-svc via the internal ALB. Internal ALB is in private app subnets — not internet-facing. ECS services use Cloud Map (service discovery) as an alternative to internal ALB for lower latency.

Internal ALB CDK

```
const internalAlb = new elbv2.ApplicationLoadBalancer(this,"InternalAlb",{
vpc,
internetFacing: false, // PRIVATE — not accessible from internet
vpcSubnets: {subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS},
securityGroups: [sgInternalAlb],
});
// Internal listener on port 8080 (HTTP — traffic stays inside VPC)
const internalListener = internalAlb.addListener("InternalHttp",{
port: 8080,
protocol: elbv2.ApplicationProtocol.HTTP,
defaultAction: elbv2.ListenerAction.fixedResponse(404,{}),
});
// /inventory/* → inventory-svc
internalListener.addTargetGroups("InventoryRule",{
priority: 10,
conditions: [elbv2.ListenerCondition.pathPatterns(["/inventory/*"])],
targetGroups: [inventoryTg],
});
```

■ Cloud Map (ECS service discovery) is an alternative to internal ALB for service-to-service calls. Cloud Map registers ECS task IPs in a private DNS namespace (internal.shopflow.local). Services call inventory-svc.internal.shopflow.local:8080 — DNS resolves to healthy task IPs without ALB hop. Use Cloud Map for lowest latency; use internal ALB when you need path-based routing, health checks with advanced rules, or connection draining during deployments.

13-13 NLB for Internal gRPC and WebSocket Traffic

Network Load Balancer (NLB) operates at Layer 4 (TCP/UDP). ShopFlow uses NLB for: (1) internal gRPC traffic between pricing-svc and inventory-svc (gRPC requires HTTP/2 end-to-end, which ALB terminates), and (2) WebSocket connections for real-time order tracking notifications (long-lived TCP connections).

Feature	ALB (Layer 7)	NLB (Layer 4)	ShopFlow Usage
Protocol	HTTP/1.1, HTTP/2, WebSocket	TCP, UDP, TLS	ALB: REST APIs. NLB: gRPC, WebSocket
Static IP	No (DNS only)	Yes (Elastic IP per AZ)	NLB when IP whitelisting required by 3PL
Latency	~1ms overhead	Sub-millisecond	NLB for lowest-latency inter-service
WAF support	Yes — WAF WebACL attachable	No WAF support	ALB for all internet-facing traffic
Health checks	HTTP/HTTPS (response code)	TCP (connection) or HTTP	Both used; ALB HTTP more precise
Connection limit	4K WebSocket per AZ	Millions of connections	NLB for WebSocket at scale

NLB CDK for Internal gRPC

```
const internalNlb = new elbv2.NetworkLoadBalancer(this,"InternalNlb",{
vpc,
internetFacing: false,
vpcSubnets: {subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS},
crossZoneEnabled: true, // distribute across AZs even if one AZ is empty
});
// gRPC listener on port 50051
internalNlb.addListener("GrpcListener",{
port: 50051,
protocol: elbv2.Protocol.TCP,
defaultTargetGroups: [grpcTargetGroup],
});
```

13-14 Route 53 Geolocation and Geo-Proximity Routing

Geolocation routing returns different records based on the geographic location of the DNS resolver (inferred as the user location). ShopFlow uses geolocation to: return the closest CDN origin, block non-US traffic for US-only product categories, and route EU users to EU-hosted privacy-compliant endpoints.

Routing Policy	Based On	ShopFlow Use Case	Fallback Behavior
Geolocation	User country or continent	EU users → eu-central-1 ALB (GDPR)	Default record if no geo match
Geoproximity	Distance with bias factor	Not used (latency routing preferred)	No automatic fallback
Latency	Actual AWS network latency from origin	Global users → fastest region	Second-lowest latency region

Geolocation Routing CDK

```
// EU users → eu-central-1 ALB (GDPR data residency)
new route53.ARecord(this,"EuGeoRecord",{
zone: hostedZone,
recordName: "api",
target: route53.RecordTarget.fromAlias(new targets.LoadBalancerTarget(euAlb)),
geoLocation: route53.GeoLocation.continent(route53.Continent.EUROPE),
setIdentifier: "eu-users",
});
// Default: all other continents → us-east-1 ALB
new route53.ARecord(this,"DefaultGeoRecord",{
zone: hostedZone,
recordName: "api",
target: route53.RecordTarget.fromAlias(new targets.LoadBalancerTarget(usAlb)),
geoLocation: route53.GeoLocation.default(),
setIdentifier: "default-users",
```

```
});
```

■ Always create a default geolocation record. Without a default, users from continents not explicitly mapped receive SERVFAIL (DNS resolution fails). The default record is the catch-all that handles all locations not covered by specific country/continent records. Geolocation routing requires a health check on each record for automatic failover if the geo-targeted endpoint fails.

13-15 ALB Capacity Planning for Black Friday

ALB scales automatically — no manual scaling required. AWS manages ALB capacity based on incoming request rate. However, ALB uses pre-warming for gradual traffic increases. For sudden 10x Black Friday traffic spike, request pre-warming from AWS Support 1-2 weeks before the event.

ALB Capacity Aspect	Normal Load	Black Friday	Action Required
Request rate	1,000 req/s	10,000 req/s	Auto-scales. Pre-warm request if spike is sudden (not gradual
Concurrent connections	50K	500K	Auto-scales. No action needed.
Bandwidth	100 Mbps	1 Gbps	Auto-scales up to 100 Gbps. No action needed.
LCU (Load Balancer Capacity Units)	~10 LCU	~100 LCU	Billed automatically per LCU-hour
Health check frequency	15s interval x 20 targets	15s interval x 200 targets (scaled ECS)	No change needed; ALB handles more targets automatically

ALB Pre-Warming Request Checklist

```
// Submit pre-warming request to AWS Support 2 weeks before Black Friday:
// 1. Expected peak requests per second: 10,000
// 2. Expected ramp-up duration: sudden (within 5 minutes of event start)
// 3. Request body average size: 2KB
// 4. Response body average size: 15KB
// 5. Estimated duration of peak: 4 hours (Black Friday sale window)
// AWS pre-warms ALB to handle 10,000 req/s from minute 1 without gradual scaling
// Without pre-warming: ALB may need 5-10 minutes to auto-scale, causing 503 errors
```

■ ALB pricing: $0.008/LCU-hour + $0.0225/hour base. LCU = max of: (new connections/s)/25, (active connections)/3000, (processed bytes in GB)/hour, (rule evaluations)/hour/1000. For 10,000 req/s Black Friday: ~40 LCU. Cost: $0.008 x 40 x 4 hours = $1.28 for the Black Friday window. ALB is extremely cost-effective — the bottleneck is ECS task count, not ALB capacity.

13-16 Route 53 Private Hosted Zones for VPC DNS

Route 53 Private Hosted Zones provide internal DNS resolution within the ShopFlow VPC. ECS services reference each other by internal DNS names (aurora.internal.shopflow, redis.internal.shopflow, opensearch.internal.shopflow) instead of hardcoded endpoints. Private zones are invisible to the public internet.

Private Hosted Zone CDK

```
const privateZone = new route53.PrivateHostedZone(this,"PrivateZone",{
zoneName: "internal.shopflow",
vpc,
});
// Aurora cluster CNAME (short internal name)
new route53.CnameRecord(this,"AuroraCname",{
zone: privateZone,
recordName: "aurora",
domainName: auroraCluster.clusterEndpoint.hostname,
ttl: Duration.seconds(30), // short TTL: Aurora failover switches endpoint in ~30s
});
// ECS service config:
spring.datasource.url=jdbc:postgresql://aurora.internal.shopflow:5432/shopflow
// ElastiCache Redis CNAME
```

```
new route53.CnameRecord(this,"RedisCname",{
zone: privateZone,
recordName: "redis",
domainName: redisCluster.attrPrimaryEndPointAddress,
ttl: Duration.seconds(60),
});
```

Internal DNS Name	Resolves To	TTL	Consumer Services
aurora.internal.shopflow	Aurora writer endpoint	30s (fast failover)	product-svc, order-svc, user-svc
aurora-ro.internal.shopflow	Aurora reader endpoint	60s	search-svc (read queries), analytics
redis.internal.shopflow	ElastiCache primary endpoint	60s	cart-svc, session-svc
opensearch.internal.shopflow	OpenSearch domain endpoint	300s	search-svc, indexer-lambda
internal-alb.internal.shopflow	Internal ALB DNS	60s	All inter-service calls

13-17 Monitoring ALB and Route 53 with CloudWatch

Key ALB CloudWatch metrics: RequestCount, HTTPCode_Target_5XX_Count, TargetResponseTime, UnHealthyHostCount, ActiveConnectionCount. Route 53 metrics: HealthCheckStatus, HealthCheckPercentageHealthy, DNSQueries. Alarms on 5XX errors and response time trigger PagerDuty escalation.

Metric	Namespace	Alert Threshold	Response Action
HTTPCode_Target_5XX_Count	AWS/ApplicationELB	>10 in 1 min	Check ECS task health; check Aurora connecti
TargetResponseTime p99	AWS/ApplicationELB	>2s for 5 min	Check downstream service (Aurora, Redis, Ope
UnHealthyHostCount	AWS/ApplicationELB	>0 for 2 min	Check ECS task logs; verify /actuator/health en
HealthCheckStatus	AWS/Route53	0 (unhealthy) for 1 check	Failover to secondary region initiated automatic
RejectedConnectionCount	AWS/ApplicationELB	>0	ALB at connection limit; request pre-warm or ch

CloudWatch Alarm CDK for ALB 5XX

```
new cloudwatch.Alarm(this,"Alb5xxAlarm",{
metric: alb.metricHttpCodeTarget(
elbv2.HttpCodeTarget.TARGET_5XX_COUNT,
{period: Duration.minutes(1), statistic: "Sum"}),
threshold: 10,
evaluationPeriods: 1,
comparisonOperator: cloudwatch.ComparisonOperator.GREATER_THAN_THRESHOLD,
treatMissingData: cloudwatch.TreatMissingData.NOT_BREACHING,
alarmDescription: "ALB 5XX errors exceed 10 per minute",
}).addAlarmAction(new cwActions.SnsAction(alertTopic));
```

■ Monitor HTTPCode_ELB_5XX separately from HTTPCode_Target_5XX. ELB 5XX = ALB itself returned error (target timeout, connection refused before reaching ECS). Target 5XX = ECS task returned 5XX. High ELB 5XX with zero Target 5XX = ECS tasks unhealthy (all removed from target group) or connection timeout. High Target 5XX with zero ELB 5XX = application error in ECS (check task logs, not ALB config).

13-18 ALB Request Routing for Blue-Green Deployments

ALB supports CodeDeploy blue/green deployments via weighted target group shifts. Traffic is gradually shifted from the blue (current) target group to green (new) target group: 10% green → validate → 50% → validate → 100% green → terminate blue. ECS CodeDeploy deployment controller automates this with automatic rollback on CloudWatch alarm.

Blue/Green Deployment Traffic Shift CDK

```
// ECS Service: use CODE_DEPLOY deployment controller
const ecsService = new ecs.FargateService(this,"ProductSvc",{
cluster,
taskDefinition,
deploymentController: {type: ecs.DeploymentControllerType.CODE_DEPLOY},
});
// CodeDeploy ECS Deployment Group with traffic shifting
const deploymentGroup = new codedeploy.EcsDeploymentGroup(this,"DG",{
service: ecsService,
blueGreenDeploymentConfig: {
blueTargetGroup: productTg, // current (blue) target group
greenTargetGroup: productTgGreen, // new (green) target group
listener: httpsListener,
testListener: testListener, // port 8443: QA tests against green
deploymentApprovalWaitTime: Duration.minutes(5),
terminationWaitTime: Duration.minutes(60), // keep blue for 1hr
},
deploymentConfig: codedeploy.EcsDeploymentConfig.CANARY_10PERCENT_5MINUTES,
autoRollback: {
failedDeployment: true,
deploymentInAlarm: true, // rollback if CloudWatch alarm fires
stoppedDeployment: true,
},
alarms: [albHealthAlarm, ecsErrorAlarm],
});
```

Deployment Config	Traffic Shift Schedule	Use Case	Risk
AllAtOnce	100% green immediately	Fast; risky	High — all users see new version
CANARY_10PERCENT_5MINUTES (ShopFlow)	10% → 5 min wait → 100%	Balanced risk/speed	Low — 10% exposure before full shift
LINEAR_10PERCENT_EVERY_1MINUTE	10%+10%+10%... per minute	Gradual 10-minute rollout	Very low — alarms stop shift early
Custom (e.g., 1% → 24hr → 100%)	Any schedule	Maximum caution	Minimal — tiny exposure for 24hr

13-19 Cross-Zone Load Balancing and AZ Awareness

Cross-zone load balancing distributes traffic evenly across all ECS tasks in all AZs, regardless of which AZ the request entered through. Without cross-zone: AZ with fewer tasks gets less traffic but the same number of requests per task. ShopFlow enables cross-zone on ALB (enabled by default — no extra charge for ALB).

Scenario	Without Cross-Zone	With Cross-Zone (ShopFlow)
3 AZs, 10 tasks each (balanced)	Even: each task gets 3.3% of traffic	Even: each task gets 3.3% of traffic
3 AZs: 10 tasks in AZ-A, 5 in AZ-B, 5 in AZ-C	AZ-A tasks: 3.3% each; AZ-B/C tasks: 6.7% each (20 tasks)	All (20 tasks): 5% each (even distribution)
AZ-A fails (0 tasks)	AZ-B and AZ-C handle 50% each normally	All tasks in AZ-B and AZ-C share all traffic evenly
NLB cross-zone	Disabled by default; 0.01/GB charge if enabled	Not applicable (ALB used for HTTP)

Confirming Cross-Zone Is Enabled

```
// CDK: cross-zone is enabled by default on ALB; verify:
const alb = new elbv2.ApplicationLoadBalancer(this,"ShopFlowAlb",{
// crossZoneEnabled defaults to true for ALB (no property needed)
// For NLB, explicitly set:
// crossZoneEnabled: true // note: 0.01/GB inter-AZ data transfer charge for NLB
});
// Verify in console: EC2 → Load Balancers → select ALB → Attributes tab
// "Cross-zone load balancing": Enabled
// Monitor per-AZ request distribution:
// CloudWatch: AWS/ApplicationELB RequestCount with AvailabilityZone dimension
```

```
// Target: each AZ receives ~33% (±5%) of total RequestCount
```

■ Cross-zone load balancing for ALB is free (no inter-AZ data transfer charge for ALB — unlike NLB). This makes it always-on for ALB. For NLB, cross-zone load balancing incurs $0.01/GB for inter-AZ traffic — cost consideration for high-throughput use cases. ShopFlow uses ALB for all HTTP traffic, so cross-zone load balancing has no incremental cost.

13-20 Route 53 Resolver and Hybrid DNS

Route 53 Resolver handles DNS queries from within the VPC. Inbound Resolver endpoints allow on-premises DNS servers to resolve AWS internal names (aurora.internal.shopflow). Outbound Resolver endpoints forward queries from VPC to on-premises DNS for corporate domains. ShopFlow uses outbound Resolver for connecting to on-premise warehouse system DNS.

Resolver Type	Direction	Use Case	ShopFlow Usage
Default VPC Resolver	VPC → Route 53	Resolves .internal, .amazonaws.com, and public DNS	All EC2 service DNS lookups (Aurora, Redis, OpenSe
Inbound Endpoint	On-premises → VPC	On-prem servers resolve AWS internal names via VPN/DX	Not used (no on-prem servers query ShopFlow VPC D
Outbound Endpoint	VPC → On-premises	VPC resolves on-premises corporate DNS names	warehouse.corp.shopflow → on-prem DNS for WMS s
Resolver Rules (forward)	VPC → specified DNS IPs	Forward specific domain queries to custom resolvers	corp.shopflow → on-prem DNS server at 10.100.0.2

Outbound Resolver Endpoint CDK

```
const outboundEndpoint = new route53resolver.CfnResolverEndpoint(this,"OutboundRes",{
direction: "OUTBOUND",
ipAddresses: [
{subnetId: privateSubnet1.subnetId}, // Resolver ENI in AZ-A
{subnetId: privateSubnet2.subnetId}, // Resolver ENI in AZ-B
],
securityGroupIds: [sgResolver.securityGroupId],
name: "shopflow-outbound-resolver",
});
// Forward rule: corp.shopflow → on-premises DNS over VPN
const fwdRule = new route53resolver.CfnResolverRule(this,"CorpFwdRule",{
domainName: "corp.shopflow",
ruleType: "FORWARD",
resolverEndpointId: outboundEndpoint.attrResolverEndpointId,
targetIps: [{ip: "10.100.0.2", port: "53"}], // on-prem DNS server IP
});
```

13-21 Lambda@Edge and CloudFront Functions for Auth

Lambda@Edge runs at CloudFront edge locations (not us-east-1). ShopFlow uses Lambda@Edge for: (1) JWT verification at edge — reject unauthenticated requests before they reach ALB, (2) A/B test cookie injection — set variant cookie on first visit, (3) image resizing on-the-fly — transform image URLs to serve WebP format. CloudFront Functions are lighter (sub-millisecond, V8 JS) for header manipulation.

Feature	Lambda@Edge	CloudFront Functions
Runtime	Node.js, Python (full Lambda)	JavaScript (V8 subset)
Max execution time	5s (viewer), 30s (origin)	1ms (viewer request/response only)
Memory	128MB – 10GB	2MB only
Network access	Yes — can call external APIs	No network access
Cost	Lambda pricing (per 100ms)	$0.10 per 1M invocations (~10x cheaper)
ShopFlow use case	JWT validation + A/B cookie	URL rewrite (/p/123 → /product?id=123)

Lambda@Edge JWT Verification

```
// Lambda@Edge: Viewer Request trigger — runs at every CloudFront edge PoP
exports.handler = async (event) => {
const request = event.Records[0].cf.request;
const authHeader = request.headers.authorization?.[0]?.value;
if (!authHeader?.startsWith("Bearer ")) {
return {status:"401",body:"Unauthorized"};
}
const token = authHeader.slice(7);
// Verify JWT signature using Cognito JWKS (cached in Lambda memory)
const payload = await verifyJwt(token, cognitoJwks);
if (!payload) return {status:"403",body:"Forbidden"};
// Add user ID header for downstream ALB/ECS
request.headers["x-shopflow-user-id"] = [{value: payload.sub}];
return request; // pass request to origin (ALB)
};
```

■ Cache the Cognito JWKS (JSON Web Key Set) in Lambda@Edge memory. JWKS is fetched from https://cognito-idp.us-east-1.amazonaws.com/POOL_ID/.well-known/jwks.json — a public endpoint. On cold start, fetch and cache JWKS as a module-level variable. On subsequent invocations, verify JWT signature using cached keys without HTTP call. Refresh cache when kid (key ID) in JWT header is not found in cached JWKS — Cognito rotates keys periodically.

13-BP Best Practices and Anti-Patterns

✓ Use Alias records (not CNAME) for root domain pointing to ALB/CloudFront — CNAME not allowed at zone apex

✓ Set ALB deletion protection to true — prevents accidental CloudFormation stack destroy from removing the load balancer

✓ Use TLS 1.3 (SslPolicy.TLS13_RES) — faster handshake (1-RTT), forward secrecy, no weak ciphers

✓ Use ACM wildcard cert (*.shopflow.com) — single cert, auto-renewed, no private key exposure

✓ Enable ALB access logs to S3 with 90-day retention — enables Athena queries for latency analysis and compliance

✓ Set deregistrationDelay to 30s on target groups — prevents HTTP 502 during ECS task shutdown

✓ Use path-based listener rules instead of per-service ALBs — reduces cost and complexity

✓ Route 53 health check intervals: 10s with 3-failure threshold = 30s RTO for DNS failover

✓ Attach WAF to ALB (not EC2/ECS) — WAF evaluates requests once at the edge, not per microservice

✓ Use internal (private) ALB for service-to-service calls — prevents microservice traffic from traversing internet

✓ Store session state in Redis (not ALB sticky cookies) — enables stateless ECS task scaling

✓ Deploy ALB in public subnets, ECS tasks in private subnets — only ALB needs internet-facing IP

■ Never use Simple routing if you need health checks — Simple routing does not support health check association

■ Do not set very low ALB health check timeouts (< 3s) on Java services — Spring Boot startup + JVM warmup can cause false unhealthy during deployment

■ Avoid using ALB for WebSocket long-lived connections at high scale — ALB has 4000 simultaneous WebSocket connection limit per AZ; use NLB for WebSocket at scale

■ Do not expose OpenSearch or Aurora endpoints directly — always route through ECS microservices via ALB for proper auth and rate limiting

---

13-QR Quick Reference and Interview Q&A

Concept	Key Value / Rule
Alias vs CNAME	Alias: points to AWS resource (ALB, CloudFront, S3). No charge for Alias DNS queries. Required for zone
Route 53 TTL	Low TTL (60s): faster failover but more DNS queries (small cost). High TTL (300s+): fewer queries, faster

Concept	Key Value / Rule
ALB vs NLB	ALB: Layer 7 (HTTP/HTTPS), path/host routing, WAF integration, sticky sessions, HTTP/2, WebSocket. N
Listener rule priority	Lower number = higher priority. Default rule (no conditions) = lowest priority. ShopFlow: /search/* at priorit
Health check thresholds	unhealthyThreshold × interval = time until task removed. 3 × 15s = 45s. healthyThreshold × interval = time
WAF rule action	ALLOW: passes request through. BLOCK: returns 403 immediately (before ECS). COUNT: logs match bu
TLS 1.3 benefits	1-RTT handshake (vs 2-RTT TLS 1.2). Forward secrecy mandatory. Removes weak cipher suites (RSA k